Detailed Line-by-Line Explanation of DL-2 Notebook

This document explains the uploaded Jupyter Notebook: **DL-2 Notebook**. Every important code line and markdown section is explained in detail. The notebook may contain: Deep Learning concepts TensorFlow/Keras implementation Data preprocessing Training and evaluation Prediction and visualization

Cell 1 (CODE)

Original Content:

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Line-by-Line Explanation:

Line	Code	Explanation
1	import matplotlib.pyplot as plt	Imports required Python libraries or modules.
2	import pandas as pd	Imports required Python libraries or modules.
3	import numpy as np	Imports required Python libraries or modules.

Cell 2 (MARKDOWN)

Original Content:

```
### Load Dataset
```

Line-by-Line Explanation:

Line	Code	Explanation
1	### Load Dataset	Markdown text describing concepts or notebook sections.

Cell 3 (CODE)

Original Content:

```
train_df = pd.read_csv('fashion-mnist_train.csv')
test_df = pd.read_csv('fashion-mnist_test.csv')
```

Line-by-Line Explanation:

Line	Code	Explanation
1	train_df = pd.read_csv('fashion-mnist_train.csv')	Handles tabular data using Pandas.
2	test_df = pd.read_csv('fashion-mnist_test.csv')	Handles tabular data using Pandas.

Cell 4 (CODE)

Original Content:

```
train_df.shape
```

Line-by-Line Explanation:

Line	Code	Explanation
1	train_df.shape	General statement or notebook content.

Cell 5 (CODE)

Original Content:

```
test_df.shape
```

Line-by-Line Explanation:

Line	Code	Explanation
1	test_df.shape	General statement or notebook content.

Cell 6 (CODE)

Original Content:

```
train_df.describe()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	train_df.describe()	General statement or notebook content.

Cell 7 (CODE)

Original Content:

```
train_df.label.unique()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	train_df.label.unique()	General statement or notebook content.

Cell 8 (CODE)

Original Content:

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```

Line-by-Line Explanation:

Line	Code	Explanation
1	class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',	Assigns values to variables or stores results.
2	'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']	General statement or notebook content.

Cell 9 (CODE)

Original Content:

```
x_train = train_df.iloc[:,1:].to_numpy()
x_train = x_train.reshape([-1,28,28,1])
x_train = x_train / 255
```

Line-by-Line Explanation:

Line	Code	Explanation
1	x_train = train_df.iloc[:,1:].to_numpy()	Performs numerical operations using NumPy.
2	x_train = x_train.reshape([-1,28,28,1])	Assigns values to variables or stores results.
3	x_train = x_train / 255	Assigns values to variables or stores results.

Cell 10 (CODE)

Original Content:

```
y_train = train_df.iloc[:,0].to_numpy()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	y_train = train_df.iloc[:,0].to_numpy()	Performs numerical operations using NumPy.

Cell 11 (CODE)

Original Content:

```
x_test = test_df.iloc[:,1:].to_numpy()
x_test = x_test.reshape([-1,28,28,1])
x_test = x_test / 255
```

Line-by-Line Explanation:

Line	Code	Explanation
1	x_test = test_df.iloc[:,1:].to_numpy()	Performs numerical operations using NumPy.
2	x_test = x_test.reshape([-1,28,28,1])	Assigns values to variables or stores results.
3	x_test = x_test / 255	Assigns values to variables or stores results.

Cell 12 (CODE)

Original Content:

```
y_test = test_df.iloc[:,0].to_numpy()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	y_test = test_df.iloc[:,0].to_numpy()	Performs numerical operations using NumPy.

Cell 13 (MARKDOWN)

Original Content:

```
### Visualization
```

Line-by-Line Explanation:

Line	Code	Explanation
1	### Visualization	Markdown text describing concepts or notebook sections.

Cell 14 (CODE)

Original Content:

```
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_train[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[y_train[i]])
plt.show()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	plt.figure(figsize=(10,10))	Creates visualizations and graphs.
2	for i in range(25):	General statement or notebook content.
3	plt.subplot(5,5,i+1)	Creates visualizations and graphs.
4	plt.xticks([])	Creates visualizations and graphs.
5	plt.yticks([])	Creates visualizations and graphs.
6	plt.grid(False)	Creates visualizations and graphs.
7	plt.imshow(x_train[i], cmap=plt.cm.binary)	Creates visualizations and graphs.
8	plt.xlabel(class_names[y_train[i]])	Creates visualizations and graphs.
9	plt.show()	Creates visualizations and graphs.

Cell 15 (MARKDOWN)

Original Content:

```
### Model Building
```

Line-by-Line Explanation:

Line	Code	Explanation
1	### Model Building	Markdown text describing concepts or notebook sections.

Cell 16 (CODE)

Original Content:

```
from keras.models import Sequential
from keras.layers import Dense,Conv2D,Flatten,MaxPooling2D,Dropout
```

Line-by-Line Explanation:

Line	Code	Explanation
1	from keras.models import Sequential	Imports required Python libraries or modules.
2	from keras.layers import Dense,Conv2D,Flatten,MaxPooling2D,Dropout	Imports required Python libraries or modules.

Cell 17 (CODE)

Original Content:

```
model = Sequential()

model.add(Conv2D(filters=64,kernel_size=(3,3),input_shape=(28,28,1),
                 activation='relu'))
model.add(MaxPooling2D(pool_size = (2,2)))
model.add(Dropout(rate=0.3))
model.add(Flatten())
model.add(Dense(units=32, activation='relu'))
model.add(Dense(units=10, activation='sigmoid'))
model.compile(loss='sparse_categorical_crossentropy',optimizer='adam',
              metrics=['accuracy'])
model.summary()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	model = Sequential()	Creates a Sequential neural network model.
3	model.add(Conv2D(filters=64,kernel_size=(3,3),input_shape=(28,28,1),	Adds a convolution layer for image feature extraction.
4	activation='relu'))	Uses ReLU activation function to introduce non-linearity.
5	model.add(MaxPooling2D(pool_size = (2,2)))	Reduces image dimensions while preserving important features.

6	model.add(Dropout(rate=0.3))	Randomly disables neurons during training to reduce overfitting.
7	model.add(Flatten())	Converts multidimensional feature maps into a single vector.
8	model.add(Dense(units=32, activation='relu'))	Adds a fully connected neural network layer.
9	model.add(Dense(units=10, activation='sigmoid'))	Adds a fully connected neural network layer.
10	model.compile(loss='sparse_categorical_crossentropy',optimizer='adam',	Configures optimizer, loss function, and evaluation metrics.
11	metrics=['accuracy'])	Assigns values to variables or stores results.
12	model.summary()	General statement or notebook content.

Cell 18 (CODE)

Original Content:

```
model.fit(x_train,y_train,epochs=50,batch_size=1200,validation_split=0.05)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	model.fit(x_train,y_train,epochs=50,batch_size=1200,validation_split=0.05)	Trains the deep learning model on the dataset.

Cell 19 (MARKDOWN)

Original Content:

```
### Evaluation
```

Line-by-Line Explanation:

Line	Code	Explanation
1	### Evaluation	Markdown text describing concepts or notebook sections.

Cell 20 (CODE)

Original Content:

```
evaluation = model.evaluate(x_test,y_test)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	evaluation = model.evaluate(x_test,y_test)	Measures model performance on testing data.

Cell 21 (CODE)

Original Content:

```
print(f"Accuracy: {evaluation[1]}")
```

Line-by-Line Explanation:

Line	Code	Explanation
1	print(f"Accuracy: {evaluation[1]}")	Displays output information.

Cell 22 (CODE)

Original Content:

```
y_probas = model.predict(x_test)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	y_probas = model.predict(x_test)	Uses trained model to make predictions.

Cell 23 (CODE)

Original Content:

```
y_pred = y_probas.argmax(axis=-1)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	y_pred = y_probas.argmax(axis=-1)	Assigns values to variables or stores results.

Cell 24 (CODE)

Original Content:

```
y_pred
```

Line-by-Line Explanation:

Line	Code	Explanation
1	y_pred	General statement or notebook content.

Cell 25 (CODE)

Original Content:

```
plt.figure(figsize=(10,10),)
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_test[i], cmap=plt.cm.binary)
#     plt.xlabel(f"True Class:{y_test[i]}")
    plt.title(f"Pred:{class_names[y_pred[i]]}")
plt.show()
```

Line-by-Line Explanation:

Line	Code	Explanation
1	plt.figure(figsize=(10,10),)	Creates visualizations and graphs.
2	for i in range(25):	General statement or notebook content.
3	plt.subplot(5,5,i+1)	Creates visualizations and graphs.
4	plt.xticks([])	Creates visualizations and graphs.
5	plt.yticks([])	Creates visualizations and graphs.
6	plt.grid(False)	Creates visualizations and graphs.
7	plt.imshow(x_test[i], cmap=plt.cm.binary)	Creates visualizations and graphs.
8	# plt.xlabel(f"True Class:{y_test[i]}")	Creates visualizations and graphs.
9	plt.title(f"Pred:{class_names[y_pred[i]]}")	Creates visualizations and graphs.
10	plt.show()	Creates visualizations and graphs.

Cell 26 (CODE)

Original Content:

```
from sklearn.metrics import classification_report
```

Line-by-Line Explanation:

Line	Code	Explanation
1	from sklearn.metrics import classification_report	Imports required Python libraries or modules.

Cell 27 (CODE)

Original Content:

```
num_classes = 10
class_names = ["class {}".format(i) for i in range(num_classes)]
cr = classification_report(y_test, y_pred, target_names=class_names)
print(cr)
```

Line-by-Line Explanation:

Line	Code	Explanation
1	num_classes = 10	Assigns values to variables or stores results.
2	class_names = ["class {}".format(i) for i in range(num_classes)]	Assigns values to variables or stores results.
3	cr = classification_report(y_test, y_pred, target_names=class_names)	Assigns values to variables or stores results.
4	print(cr)	Displays output information.

Overall Summary:
This notebook demonstrates important Deep Learning workflows including: Dataset preparation Model building Training neural networks Performance evaluation Prediction generation It helps understand practical implementation of deep learning using Python.